

Fair Division of Indivisible Goods

A Technical Report Submitted
in Partial Fulfillment of the Requirements
for the Degree of

Master of Technology
in
Computing and Mathematics

by
Saurabh Kumar
(112402005)



**DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING**
INDIAN INSTITUTE OF TECHNOLOGY PALAKKAD

May 2026

Certificate

This is to certify that the work contained in the project entitled "*Fair Division of Indivisible Goods*" is a bonafide work of *Saurabh kumar (Roll No. 112402005)*, carried out in the Department of Computer Science and Engineering, *Indian Institute of Technology Palakkad* under my guidance and that it has not been submitted elsewhere for a degree.

Dr. Pratik Ghosal

Assistant/Associate Professor

Department of Computer Science & Engineering

Indian Institute of Technology Palakkad

Acknowledgements

I would like to express my sincere gratitude to all those who have supported me in completing this project on *“Fair Division of Indivisible Goods”*. I am deeply grateful to my project guide, *Dr. Pratik Ghosal*, for his invaluable guidance, expert advice, and constant encouragement throughout this endeavor. His insights were crucial to the successful completion of this work. I also extend my thanks to the *Department of Computer Science* at *Indian Institute of Technology Palakkad* for providing the necessary resources. My heartfelt thanks to my friends for their support and to my family for their unwavering encouragement. Finally, I acknowledge all the researchers whose work served as a foundation for this project.

Saurabh kumar

Contents

1	Introduction	8
1.1	Introduction	8
2	The Settings	10
2.1	Key Fairness Notions	10
2.1.1	Envy-Freeness	10
2.1.2	Envy-Freeness up to One Good (EF1)	10
2.1.3	Envy-Freeness up to Any Good (EFX)	10
2.1.4	Maximum Nash Welfare (MNW)	10
2.2	Classical Algorithms	11
2.2.1	Round-Robin Algorithm	11
2.2.2	Envy-Cycle Elimination Algorithm	11
2.2.3	EFX for Identical Valuations: Leximin++	12
2.2.4	EFX for Two and Three Agents	12
3	Graph-Constrained EFX	13
3.1	Hidden Envy and Graph-Based Fairness	13
3.1.1	Hidden Envy (HEF-k)	13
3.1.2	Uniform Hidden Envy (uHEF-k)	13
3.1.3	Basic Observations	13
3.1.4	Graph-Based Hidden Envy	14
3.1.5	Neighborhood in Graph	14
3.2	Envy-Free up to Any Good (EFX)	14
3.2.1	Strong Envy	14
3.2.2	EFX Allocation	14
3.2.3	Graph-Based EFX	15
3.2.4	Observations	15
3.2.5	Special Cases Where EFX Exists	15
3.3	Chores and Mixed Items in Fair Division	15
3.3.1	Chore Division Model	15
3.3.2	Strong Envy in Chores	16
3.3.3	EFX for Chores	16
3.3.4	Mixed Goods and Chores	16
3.3.5	Strong Envy in Mixed Settings	16
3.3.6	EFX Allocation in Mixed Settings	16
3.3.7	Theoretical Insight	17
3.4	EFX on Stars and Generalizations	17
3.4.1	Star Graph	17
3.4.2	EFX on Star Graphs	17
3.4.3	Core-Outer Generalization	17

3.4.4	Special Case	18
3.4.5	Extension to Additive Valuations	18
3.4.6	Example	18
3.4.7	Complexity Result	18
3.5	EFX on Path	19
3.5.1	Path Graph Structure	19
3.5.2	EFX Existence on P_4	19
3.5.3	Generalization of Path Structure	19
3.5.4	General EFX Result	20
3.5.5	Example	20
3.6	EFX for Mixed Goods and Chores	20
3.6.1	Basic Setting	20
3.6.2	EFX in Chore-Only Instances	21
3.6.3	Graph-Based EFX with Chores	21
3.6.4	Limitations in Mixed Settings	21
3.6.5	Lexicographic Valuations	21
3.6.6	Graph Diameter and Fairness	21
3.6.7	EFX Guarantee in Large Graphs	21
3.6.8	Illustration	22
3.6.9	Example	22
4	Sweeping Algorithm for Path Graphs	23
4.1	Overview	23
4.1.1	Sweeping Algorithm (Overview)	23
4.1.2	Local EFX for Two Agents	23
4.1.3	The Sweeping Algorithm Pseudocode	24
4.2	Flowchart of Sweeping Algorithm	25
4.3	Python Implementation	25
4.3.1	Correctness Analysis	29
4.4	Potential Functions	29
5	Tree-Based EFX Allocation Algorithm (Proposed)	30
5.1	Motivation	30
5.2	Algorithm Description	30
5.3	Example of Tree-Based EFX Allocation with Different Valuations	31
5.3.1	Tree Structure	31
5.3.2	Goods and Valuations	31
5.3.3	Step 1: Initial Allocation	32
5.3.4	Step 2: Forward Sweep	32
5.3.5	Step 3: Backward Sweep	32
5.3.6	Final Allocation (Example Outcome)	32
5.4	Flowchart of Tree-based EFX Allocation Algorithm	33
5.5	Python Implementation	33
5.6	Correctness	39
5.6.1	Termination Condition	39
6	Conclusion	41
6.1	Summary of Work	41
6.2	Key Findings	41
6.3	Limitations and Open Problems	42

List of Algorithms

1	Round-Robin Algorithm [5]	11
2	Envy-Cycle Elimination Algorithm [5]	11
3	EFX Allocation for Agent 1	12
4	Local EFX for Two Agents [6]	24
5	Sweeping Algorithm [6]	24
6	Tree-based EFX Allocation Algorithm	31

List of Tables

4.1	Potential functions used to analyze convergence of EFX-type algorithms [1]	29
-----	--	----

Listings

4.1	Sweeping Algorithm Implementation for Path Graphs	25
5.1	Tree-Based EFX Allocation Algorithm	33

Chapter 1

Introduction

1.1 Introduction

Dividing indivisible goods in a fair manner is an important problem in areas such as algorithmic economics and multi-agent systems. It appears in many real-life situations, including assigning computational resources, distributing inheritance, and allocating courses to students. The main difficulty arises because the items cannot be split, and different agents may value them differently. As a result, simple allocation methods often fail to ensure fairness.

In this report, we focus on a strong fairness concept known as **EFX (Envy-Freeness up to Any Good)**. An allocation is said to satisfy EFX if no agent prefers another agent's bundle, even after removing any single item from that bundle. Although EFX allocations are known to exist for small cases such as two or three agents, whether they always exist in general settings is still an open problem [6, 7].

To address this challenge, we study a relaxed version of the problem called **graph-constrained EFX**. In this setting, agents are connected through a graph, and fairness is only required between neighboring agents. This restriction reflects many practical scenarios where agents only compare themselves with a limited group, such as friends, colleagues, or nearby nodes in a network.

The report is divided into two main parts. In the first part, we introduce the basic concepts needed to understand the problem, including additive valuations and fairness notions such as EF1, EFX, and Maximum Nash Welfare (MNW). We also review standard algorithms like Round-Robin and Envy-Cycle Elimination, which guarantee weaker fairness properties.

In the second part, we explore existing research on graph-based fairness [3]. We discuss different models such as hidden envy, settings with both goods and chores, and known results for specific graph structures like stars, paths, and core-outer graphs. These results show that restricting fairness to a graph can make the problem more manageable.

The main contributions of this work are two algorithms designed for graph-constrained settings:

- A **Sweeping Algorithm** for path graphs, which repeatedly adjusts allocations between neighboring agents until no strong envy remains.
- A **Tree-Based EFX Algorithm**, which extends this idea to tree structures using depth-first traversal and local fairness adjustments.

We analyze the correctness of both algorithms and study their behavior using potential functions to understand termination. We also provide Python implementations and

examples. Overall, this work demonstrates that by using graph structures, it is possible to achieve strong fairness guarantees even when the general problem remains unresolved.

Chapter 2

The Settings

We consider a set N of n agents and a set M of m indivisible goods. Each agent i has an additive valuation function $v_i : 2^M \rightarrow \mathbb{R}_{\geq 0}$. An allocation $A = (A_1, \dots, A_n)$ assigns disjoint bundles to all agents.

An **envy graph** $G = (N, E)$ specifies which agent pairs can compare bundles. The **Graph-EFX** condition requires: for every edge $(i, j) \in E$ and every good $g \in A_j$,

$$v_i(A_i) \geq v_i(A_j \setminus \{g\}).$$

2.1 Key Fairness Notions

2.1.1 Envy-Freeness

Definition 1. An allocation A is said to be *envy-free (EF)* if every agent values their own bundle at least as much as the bundle of any other agent. Formally,

$$v_i(A_i) \geq v_i(A_j) \quad \forall i, j \in N.$$

2.1.2 Envy-Freeness up to One Good (EF1)

Definition 2. An allocation A is *EF1* if for every pair $i, j \in N$, there exists $g \in A_j$ such that

$$v_i(A_i) \geq v_i(A_j \setminus \{g\})$$

.

2.1.3 Envy-Freeness up to Any Good (EFX)

Definition 3. An allocation A is *EFX* if for every pair $i, j \in N$ and every $g \in A_j$,

$$v_i(A_i) \geq v_i(A_j \setminus \{g\})$$

.

2.1.4 Maximum Nash Welfare (MNW)

Definition 4. An allocation A maximizes Nash welfare if it maximizes $\prod_{i: v_i(A_i) > 0} v_i(A_i)$ among allocations maximizing the number of agents with positive utility.

2.2 Classical Algorithms

2.2.1 Round-Robin Algorithm

Algorithm 1 Round-Robin Algorithm [5]

```

1: Input: A fair allocation instance  $I = \langle N, M, v \rangle$  with  $n$  agents and  $m$  goods.
2: Output: Allocation  $A = (A_1, \dots, A_n)$ .
3: for each agent  $i \in N$  do
4:    $A_i \leftarrow \emptyset$ ;
5: end
6: for  $t = 1, \dots, m$  do
7:   Let  $i \leftarrow t \bmod n$ ;
8:   Let  $g^* \in \arg \max_{g \in M} v_i(g)$ ;
9:    $A_i \leftarrow A_i \cup \{g^*\}$ ;
10:   $M \leftarrow M \setminus \{g^*\}$ ;
11: end

```

Theorem 1 (Round-Robin Guarantees EF1 [5]). *For additive valuations, the Round-Robin algorithm always produces an EF1 allocation.*

Proof. In Round-Robin, agents pick items one by one in turns. If an agent i envies another agent j , it can only be because j received some item that i preferred.

If we remove the last item assigned to j , then all remaining items in j 's bundle were either less preferred by i or were not available when i chose. Hence, i no longer envies j .

Therefore, the allocation satisfies EF1. $\square$

2.2.2 Envy-Cycle Elimination Algorithm

Algorithm 2 Envy-Cycle Elimination Algorithm [5]

```

1: Input: A fair allocation instance  $I = \langle N, M, v \rangle$  with  $n$  agents and  $m$  goods.
2: Output: Allocation  $A = (A_1, \dots, A_n)$ .
3: for each agent  $i \in N$  do
4:    $A_i \leftarrow \emptyset$ ;
5: end
6: for  $t = 1, \dots, m$  do
7:   while there does not exist an unenvied agent do
8:     Find an envy-cycle  $C = (i_1, \dots, i_d)$  and resolve the cycle as follows:
9:     
$$A_i^C = \begin{cases} A_{i_{j+1}} & \text{for all } 1 \leq j \leq d-1 \\ A_{i_1} & \text{for } j = d \end{cases} \quad (1)$$

10:     $A_i \leftarrow A_i^C$  for all  $i \in C$ ;
11:   end
12:   Let  $i$  be an unenvied agent;
13:   Let  $g^* \in \arg \max_{g \in M} v_i(g)$ ;
14:    $A_i \leftarrow A_i \cup \{g^*\}$ ;
15:    $M \leftarrow M \setminus \{g^*\}$ ;
16: end

```

Theorem 2 (Envy-Cycle Elimination [5]). *The Envy-Cycle Elimination algorithm runs in polynomial time and always produces an EF1 allocation.*

Proof. In each step, the algorithm either assigns a good to an unenvied agent or removes an envy-cycle by rotating bundles among agents. Removing a cycle does not create new envy and strictly improves the allocation structure.

Since there are only finitely many goods, and each good is allocated exactly once, the algorithm completes in polynomial time.

Moreover, any remaining envy between agents can be eliminated by removing at most one good from the other agent's bundle. Hence, the final allocation satisfies EF1. $\square$

2.2.3 EFX for Identical Valuations: Leximin++

Theorem 3 (Plaut and Roughgarden [6]). *For agents with identical monotone valuations, a leximin++ allocation satisfies EFX.*

2.2.4 EFX for Two and Three Agents

Theorem 4 (Plaut and Roughgarden [6]; Chaudhury et al. [7]). *EFX allocations always exist for two agents (using a cut-and-choose method) and for three agents (using a pseudo-polynomial time algorithm).*

Algorithm 3 EFX Allocation for Agent 1

```

1: Input: Goods  $g_1, g_2, \dots, g_m$  with values for agent 1
2: Sort the goods in decreasing order of value for agent 1
3:  $X_1 \leftarrow \emptyset, \quad X_2 \leftarrow \emptyset$ 
4: for each good  $g$  in sorted order do
5:   if  $v_1(X_1) \leq v_1(X_2)$  then
6:      $X_1 \leftarrow X_1 \cup \{g\}$ 
7:   else
8:      $X_2 \leftarrow X_2 \cup \{g\}$ 
9:   end if
10: end for
11: return  $(X_1, X_2)$ 

```

Chapter 3

Graph-Constrained EFX

This chapter examines the key ideas from [2], where the authors proposed relaxing the envy-freeness requirement by only requiring fairness between neighboring agents in a graph.

3.1 Hidden Envy and Graph-Based Fairness

In fair division problems, we often want to ensure that no agent feels envious of another agent's allocation. However, achieving complete envy-freeness is not always possible. To address this, we allow the possibility of *hiding* a small number of goods [9].

3.1.1 Hidden Envy (HEF- k)

An allocation X is said to be **envy-free up to k hidden goods (HEF- k)** if there exists a subset of goods $S \subseteq M$ with $|S| \leq k$ such that, after removing these goods from consideration, no agent envies another. Formally, for every pair of agents $i, j \in N$:

$$v_i(X_i) \geq v_i(X_j \setminus S)$$

In simple terms, we are allowed to "ignore" up to k goods so that the remaining allocation appears fair to everyone [9].

3.1.2 Uniform Hidden Envy (uHEF- k)

An allocation is called **uniformly envy-free up to k hidden goods (uHEF- k)** if it satisfies HEF- k , and additionally, each agent has at most one of their own goods hidden. That is:

$$|S \cap X_i| \leq 1 \quad \text{for all } i \in N$$

This condition ensures that the burden of hiding goods is distributed fairly among agents [9].

3.1.3 Basic Observations

- Every uHEF- k allocation guarantees envy-freeness up to one good (EF1) [5].
- For additive valuations, a uHEF- $(n - 1)$ allocation always exists, for example using a round-robin method.

- However, uHEF- k allocations for smaller k may not exist. For instance, if all agents value goods identically and there are only $n-1$ goods, one agent will receive nothing, making it necessary to hide all goods to eliminate envy [9].

3.1.4 Graph-Based Hidden Envy

Sometimes, agents are connected through a graph $G = (N, E)$, where edges represent relationships (e.g., visibility or comparison). In such cases, we only require fairness between neighboring agents.

An allocation X is called **G -HEF- k** if there exists a subset $S \subseteq M$, $|S| \leq k$, such that for every edge $(i, j) \in E$:

$$v_i(X_i) \geq v_i(X_j \setminus S)$$

If, in addition, each agent has at most one hidden good from their own bundle, then the allocation is called **G -uHEF- k** [9].

3.1.5 Neighborhood in Graph

For any agent i , its neighborhood in graph G is defined as:

$$\text{Nbd}_G(i) = \{j \in N \mid (i, j) \in E\}$$

This represents all agents directly connected to i .

3.2 Envy-Free up to Any Good (EFX)

A stronger notion of fairness than EF1 is called **Envy-Free up to Any Good (EFX)**. This concept requires that envy disappears even after removing any single good from another agent's bundle [5].

3.2.1 Strong Envy

Consider an allocation X . If agent i prefers agent j 's bundle over their own, i.e.,

$$v_i(X_j) > v_i(X_i),$$

then i envies j .

We say that agent i has **strong envy** towards agent j if there exists some good $g \in X_j$ such that even after removing g , agent i still prefers j 's bundle:

$$v_i(X_j \setminus \{g\}) > v_i(X_i).$$

The amount of strong envy can be written as:

$$\max \left\{ \max_{g \in X_j} (v_i(X_j \setminus \{g\}) - v_i(X_i)), 0 \right\}.$$

3.2.2 EFX Allocation

An allocation X is called **EFX** if no agent strongly envies another. Formally, for all agents $i, j \in N$ and every good $g \in X_j$:

$$v_i(X_i) \geq v_i(X_j \setminus \{g\}).$$

This ensures that removing any single good from another agent's bundle eliminates envy [6].

3.2.3 Graph-Based EFX

Now suppose agents are connected through a graph $G = (N, E)$. In this setting, fairness is only required between neighboring agents.

An allocation X is called **G -EFX** if for every edge $(i, j) \in E$:

$$v_i(X_i) \geq v_i(X_j \setminus \{g\}) \quad \text{for all } g \in X_j.$$

Thus, we only enforce the EFX condition locally along the edges of the graph.

3.2.4 Observations

- If the graph G has multiple connected components, the problem can be solved independently on each component. Combining these solutions gives a valid G -EFX allocation.
- If the graph contains at most three agents, an EFX allocation is guaranteed to exist under additive valuations. This is known for complete graphs on two and three vertices [6, 7].
- Therefore, it is sufficient to focus on connected graphs with at least four agents.

3.2.5 Special Cases Where EFX Exists

EFX allocations are known to exist in several restricted settings:

- When all agents have identical valuations
- When there are only two types of valuation functions
- When each good has only two possible values
- When valuations are submodular with binary marginal gains
- When preferences are lexicographic

These results are established in prior works [6, 8, 9, 11, 12].

3.3 Chores and Mixed Items in Fair Division

So far, we have discussed fair allocation of goods (items with positive value). We now consider the setting where the items are **chores**, i.e., undesirable tasks that agents prefer to avoid. This setting requires careful adaptation of fairness concepts [9].

3.3.1 Chore Division Model

An instance of chore division consists of:

- A set of agents $N = \{1, 2, \dots, n\}$
- A set of chores $M = \{c_1, c_2, \dots, c_m\}$

Each agent i has a valuation function:

$$v_i : 2^M \rightarrow \mathbb{R}_{\leq 0}$$

which assigns a *non-positive* value to every subset of chores. This reflects the fact that chores are undesirable.

3.3.2 Strong Envy in Chores

Let X be an allocation. If agent i prefers the bundle of agent j over their own, we say that i envies j .

In the case of chores, we define **strong envy** differently. Agent i is said to strongly envy agent j if there exists a chore $c \in X_i$ such that removing c from i 's bundle still leaves i worse off than j :

$$v_i(X_j) > v_i(X_i \setminus \{c\})$$

The amount of strong envy can be expressed as:

$$\max \left\{ \max_{c \in X_i} (v_i(X_j) - v_i(X_i \setminus \{c\})), 0 \right\}$$

3.3.3 EFX for Chores

An allocation is said to be **EFX for chores** if no agent strongly envies another. In other words, for every pair of agents $i, j \in N$, there exists no chore in i 's bundle whose removal still leaves i envious of j [6].

3.3.4 Mixed Goods and Chores

In many real-world situations, we may have both desirable items (goods) and undesirable items (chores). Let:

$$M = \{o_1, o_2, \dots, o_m\}$$

be the set of all items, partitioned into:

- M^+ : set of goods
- M^- : set of chores

We assume that each item is either a good for all agents or a chore for all agents.

3.3.5 Strong Envy in Mixed Settings

In the presence of both goods and chores, agent i strongly envies agent j if either of the following holds:

- There exists a good $g \in X_j$ such that removing g still leaves i envious:

$$v_i(X_j \setminus \{g\}) > v_i(X_i)$$

- There exists a chore $c \in X_i$ such that removing c from i 's bundle still leaves i envious:

$$v_i(X_j) > v_i(X_i \setminus \{c\})$$

The total amount of strong envy is given by:

$$\max \left\{ \max_{g \in X_j} (v_i(X_j \setminus \{g\}) - v_i(X_i)), \max_{c \in X_i} (v_i(X_j) - v_i(X_i \setminus \{c\})), 0 \right\}$$

3.3.6 EFX Allocation in Mixed Settings

An allocation is considered **EFX** if no strong envy exists between any pair of agents. This definition naturally extends to both goods and chores [6, 9].

3.3.7 Theoretical Insight

Theorem. In a fair division instance involving goods, chores, or both, an allocation that eliminates all strong envy among agents satisfies the EFX fairness criterion.

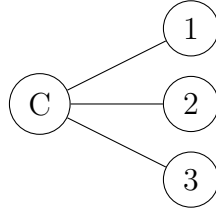
Discussion. This result follows directly from the definition of strong envy. If no agent strongly envies another, then removing any single good (in case of goods) or removing any single chore (in case of chores) is sufficient to eliminate envy. Hence, the allocation meets the EFX condition.

3.4 EFX on Stars and Generalizations

In this section, we study how to achieve EFX allocations when agents are connected in a star-shaped graph and its extensions. These structures are useful because they simplify fairness requirements by restricting comparisons between agents [3].

3.4.1 Star Graph

Definition. A star graph $K_{1,n-1}$ consists of one central agent connected to all other agents, while the outer agents are not connected to each other.



Here, agent C is the center and all other agents are outer agents.

3.4.2 EFX on Star Graphs

Theorem 1. For any number of agents arranged as a star graph, there exists an allocation that satisfies the G -EFX property under general valuations.

Idea. We first compute an EFX allocation assuming all agents have the same valuation as the center [6]. This produces multiple bundles. Then:

- Outer agents pick bundles first
- The remaining bundle is assigned to the center

This ensures that no agent strongly envies another along any edge.

3.4.3 Core-Outer Generalization

The star graph idea can be extended to more general structures.

Definition (Core-Outer Structure). Let the set of agents N be divided into:

- A **core set** N^0 where all agents have identical valuation functions
- An **outer set** $N \setminus N^0$ where agents may have different valuations

The outer agents:

- Can connect to any core agents

- Do not have edges among themselves (independent set)

Theorem 2. If a graph follows the core-outer structure described above, then a G -EFX allocation always exists.

Idea. We compute an allocation for identical core agents [6]. Then outer agents select bundles first, ensuring fairness with respect to their neighbors.

3.4.4 Special Case

Corollary. If there are four agents and at least two of them have identical valuations, then an allocation exists that is EFX for almost all agent pairs.

Explanation. The identical agents form the core, and the remaining agents act as outer nodes.

3.4.5 Extension to Additive Valuations

We can relax the requirement of identical valuations.

Definition (Consistent Valuations). Valuations are called consistent if agents agree on the ranking of goods, even if their exact values differ.

Theorem 3. Under additive and consistent valuations, if each outer agent is connected only to a group of core agents with identical valuation functions, then a G -EFX allocation exists.

Idea. We temporarily assume that outer agents have the same valuation as their neighbors and compute an allocation [6]. Then outer agents choose bundles first.

3.4.6 Example

Consider three agents $\{1, 2, 3\}$ and six goods $\{g_1, \dots, g_6\}$:

Agent	g_1	g_2	g_3	g_4	g_5	g_6
1	9	1	0	0	0	0
2	8	0	0	0	0	2
3	2	2	2	1	2	1

Suppose agents 1 and 3 have similar preferences, but agent 2 differs. If we incorrectly assign agent 2 the same behavior as another agent, the allocation may fail to satisfy EFX.

This example shows that careful handling of valuation differences is necessary.

The above show that EFX allocations can be guaranteed in structured graphs such as stars and core-outer graphs. However, extending these results to arbitrary graphs is challenging due to conflicts in agent preferences.

3.4.7 Complexity Result

Theorem 4. For additive valuations, G -EFX allocations in the above graph classes can be computed efficiently in polynomial time.

Interpretation. Although computing EFX is hard in general, structured graphs allow practical algorithms [6].

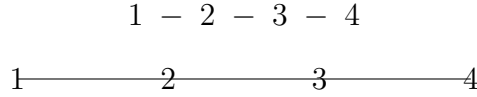
3.5 EFX on Path

In this section, we analyze the existence of EFX allocations when agents are arranged in a path graph. Path structures are important because they represent sequential relationships where each agent only compares with immediate neighbors.

3.5.1 Path Graph Structure

Definition. A path graph P_n is a graph where agents are arranged in a linear sequence such that each agent is connected only to its immediate neighbors.

A simple case is the three-edge path P_4 :



3.5.2 EFX Existence on P_4

Proposition. For agents arranged in a three-edge path P_4 , there always exists an allocation that satisfies the G -EFX property, even when agents have general valuation functions.

Idea of Proof. We divide the agents into two groups:

- Agents $\{1, 2\}$ follow one valuation pattern
- Agents $\{3, 4\}$ follow another valuation pattern

Using known results for two valuation types [1], we first compute an allocation:

$$X = (X_1, X_2, X_3, X_4)$$

Then:

- Agent 1 chooses between X_1 and X_2
- Agent 3 chooses between X_3 and X_4

This ensures that envy is removed along each edge.

3.5.3 Generalization of Path Structure

The idea from P_4 can be extended to more complex graphs derived from path structures.

Definition (Two-Type Core Path Structure). Let the agent set N be divided into:

- A **core set** N^0 consisting of agents with two types of valuation functions
- An **outer set** $N \setminus N^0$ consisting of agents with arbitrary valuations

The core is partitioned as:

$$N^0 = N_k^0 \cup N_\ell^0$$

Each outer agent:

- Is connected only to one of these two groups
- Has no connections with other outer agents

3.5.4 General EFX Result

Theorem. If a graph follows the above path-based structure with two valuation types in the core and restricted neighborhoods for outer agents, then a G -EFX allocation is guaranteed to exist.

Idea of Proof. We use a two-valuation allocation method [1]:

- Assume outer agents temporarily adopt the valuation of their neighbors
- Compute an allocation for this simplified setting
- Allow outer agents to pick bundles before core agents

This ensures no strong envy remains between connected agents.

3.5.5 Example

Consider four agents connected as:

$$1 - 2 - 3 - 4$$

Let the goods be $\{g_1, g_2, g_3, g_4\}$.

Assume:

- Agents 1 and 2 value goods similarly
- Agents 3 and 4 follow a different preference pattern

One possible allocation:

$$X = (\{g_1\}, \{g_2\}, \{g_3\}, \{g_4\})$$

By letting agents 1 and 3 choose first within their groups, the allocation satisfies EFX along all edges.

Path graphs provide a useful structure for designing fair allocation algorithms. By grouping agents based on valuation similarities and controlling the order of allocation, we can guarantee EFX even in cases with general valuations.

These results also extend to slightly more complex graphs such as longer paths or hybrid structures [6, 8].

3.6 EFX for Mixed Goods and Chores

In many real-world situations, agents must divide both desirable items (goods) and undesirable items (chores). This mixed setting makes fairness more challenging, since agents prefer to receive goods but avoid chores. In this section, we study the existence of EFX allocations in such environments, especially when agents are connected through a graph.

3.6.1 Basic Setting

Let M be the set of all items, divided into:

- M^+ : set of goods (positive value)
- M^- : set of chores (negative value)

Each agent i has an additive valuation function v_i over subsets of M .

3.6.2 EFX in Chore-Only Instances

We first consider the special case where all items are chores.

Observation. When agents have additive and consistent valuations, an EFX allocation always exists for chores and can be computed efficiently [4].

3.6.3 Graph-Based EFX with Chores

Using the above result, we extend fairness guarantees to graph structures.

Theorem 1 [4]. Let all items be chores and valuations be additive. If the graph $G = (N, E)$ contains:

- A core group of agents with identical valuations
- Outer agents forming an independent set

then a G -EFX allocation exists and can be computed in polynomial time.

Theorem 2 [4]. Let the core group be partitioned into subsets where each subset has identical valuations. If every outer agent is connected only to one such subset, then a G -EFX allocation exists and is efficiently computable.

3.6.4 Limitations in Mixed Settings

When both goods and chores are present, EFX allocations are not always guaranteed to exist for general valuations [10]. Even under structured preferences such as lexicographic valuations, existence may fail [9].

3.6.5 Lexicographic Valuations

Definition. A valuation is called **lexicographic** if each agent has a strict priority ordering over items:

$$o_1 \succ o_2 \succ \cdots \succ o_m$$

Agents compare bundles based on the highest-priority item they contain (for goods) or avoid (for chores).

3.6.6 Graph Diameter and Fairness

Definition. The **diameter** of a graph is the maximum shortest-path distance between any two agents.

3.6.7 EFX Guarantee in Large Graphs

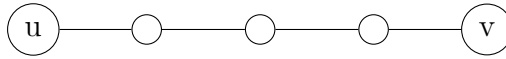
Theorem 3. If a graph has diameter at least 4, then a G -EFX allocation exists for mixed goods and chores under lexicographic valuations, and it can be computed in polynomial time.

Idea of Proof. We select two agents that are far apart in the graph. Then:

- Neighbors of one agent are assigned goods
- Neighbors of the other are assigned chores
- Remaining items are distributed between the two distant agents

This separation prevents strong envy between neighboring agents.

3.6.8 Illustration



Here, agents u and v are far apart (distance ≥ 4), which allows us to distribute goods and chores without creating envy along edges.

3.6.9 Example

Consider five agents connected in a path:

$$1 - 2 - 3 - 4 - 5$$

Let:

- Goods: $\{g_1, g_2\}$
- Chores: $\{c_1, c_2\}$

A possible allocation:

- Agent 2 receives goods
- Agent 4 receives chores
- Agents 1 and 5 receive remaining items
- Agent 3 receives no items

This allocation satisfies EFX along all edges because no agent strongly envies its neighbors.

Although EFX may not exist in general mixed settings, restricting comparisons using graph structures can restore fairness. In particular, large-diameter graphs such as trees and sparse networks often admit EFX allocations.

This shows that graph-based models are powerful tools for achieving fairness in complex allocation problems [9].

Chapter 4

Sweeping Algorithm for Path Graphs

4.1 Overview

4.1.1 Sweeping Algorithm (Overview)

The Sweeping Algorithm is used to compute an EFX allocation when agents are arranged on a path graph. Initially, all goods are assigned to the first agent. The algorithm then improves fairness step by step through multiple rounds.

In each round, the algorithm performs two passes along the path: a forward sweep (from left to right) and a reverse sweep (from right to left). At each step, it considers a pair of neighboring agents connected by an edge, say $(i, i + 1)$, and redistributes the goods currently held by them.

This redistribution ensures that neither agent strongly envies the other. It is achieved using a two-agent fair division method based on the cut-and-choose principle, which is known to guarantee EFX for two agents [6].

Fixing fairness between one pair of agents may disturb fairness with adjacent agents. Therefore, the algorithm repeatedly sweeps across the graph until all neighboring pairs satisfy the EFX condition.

Overall, the algorithm works by locally resolving envy along edges and gradually converges to a globally fair allocation on the path graph.

4.1.2 Local EFX for Two Agents

For the case of two agents i and j with additive valuation functions, it is always possible to compute an EFX allocation using a simple and intuitive procedure.

One common approach is based on the **cut-and-choose** idea. In this method, one agent (say i) first divides the goods into two bundles in a balanced way according to their own valuation. Then, the other agent (j) selects the bundle they prefer. This ensures that agent j does not envy i , while agent i has already divided the goods fairly from their own perspective.

Another practical method, inspired by [6], works as follows: the goods are first ordered in decreasing value according to agent i . Then, items are assigned one by one to the bundle that currently has lower value for agent i , keeping the two bundles as balanced as possible. Finally, agent j chooses their preferred bundle.

Both approaches guarantee that the resulting allocation satisfies the EFX property for two agents.

Algorithm 4 Local EFX for Two Agents [6]

Require: Bundles X_1, X_{n+1} ; additive valuations v_1, v_{n+1}

- 1: $M' \leftarrow X_1 \cup X_{n+1}$
 - 2: Sort M' in decreasing order of v_{n+1}
 - 3: $Y_1 \leftarrow \emptyset, Y_2 \leftarrow \emptyset$
 - 4: **for** each good $g \in M'$ **do**
 - 5: **if** $v_{n+1}(Y_1) \leq v_{n+1}(Y_2)$ **then**
 - 6: $Y_1 \leftarrow Y_1 \cup \{g\}$
 - 7: **else**
 - 8: $Y_2 \leftarrow Y_2 \cup \{g\}$
 - 9: **end if**
 - 10: **end for**
 - 11: $X_{n+1} \leftarrow \arg \max_{Y \in \{Y_1, Y_2\}} v_{n+1}(Y)$
 - 12: $X_1 \leftarrow M' \setminus X_{n+1}$
 - 13: **return** (X_1, X_{n+1})
-

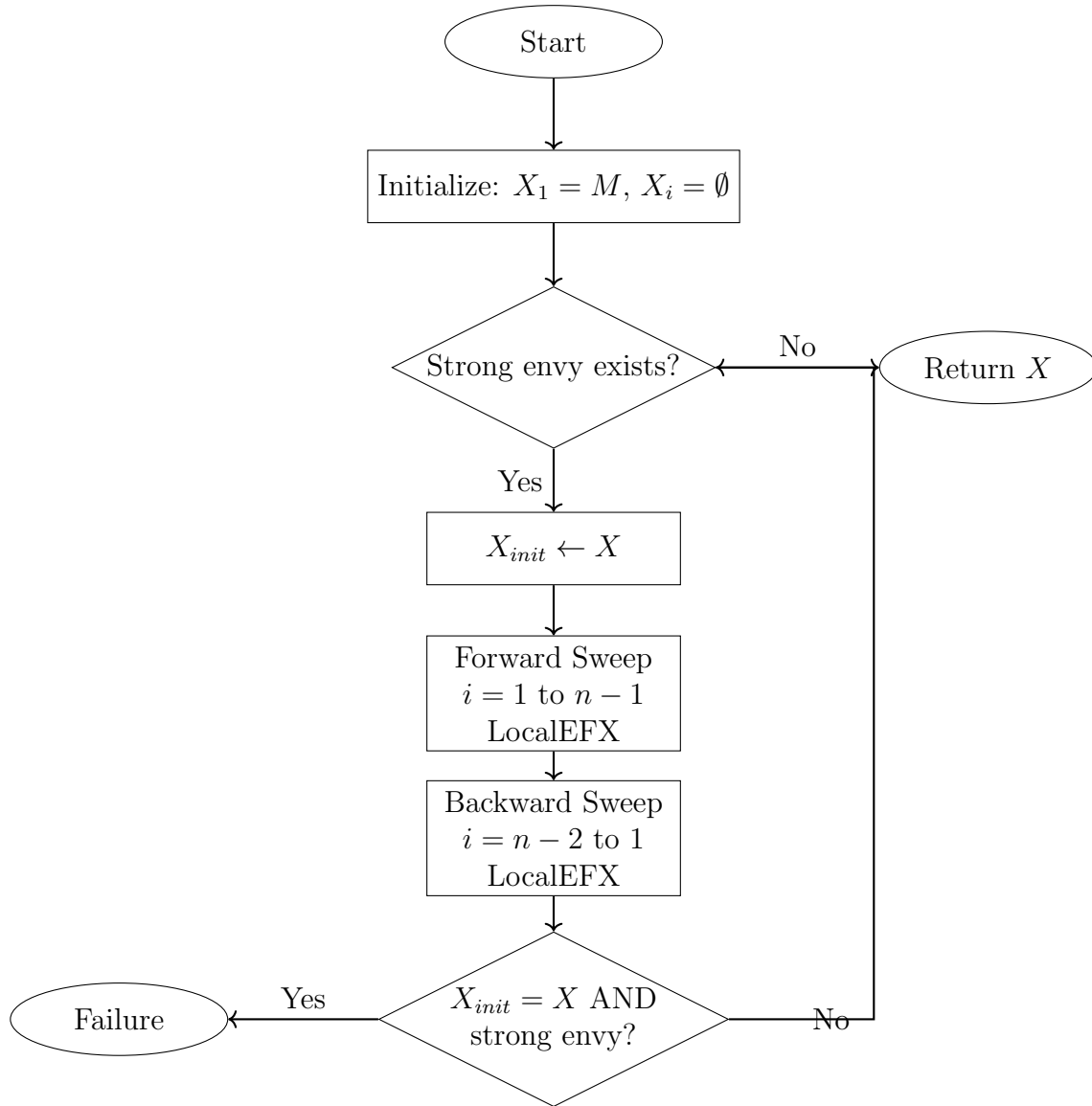
4.1.3 The Sweeping Algorithm Pseudocode

Algorithm 5 Sweeping Algorithm [6]

Require: n agents N , m goods M , additive valuation functions $v_i : 2^M \rightarrow \mathbb{R}$ for agents $i \in N$, path graph P_n

- 1: Initialize allocation $X = (X_1, \dots, X_n)$ with $X_1 = M$ and $X_i = \emptyset$ for all $i \neq 1$
 - 2: **while** there exists an edge $(i, i+1)$ in P_n such that i strongly envies $i+1$ or $i+1$ strongly envies i **do**
 - 3: $X^{\text{init}} \leftarrow X$
 - 4: **for** $i \in \{1, \dots, n-1\}$ **do**
 - 5: $(X_i, X_{i+1}) \leftarrow \text{LOCALEFX}(X_i, X_{i+1}, v_i, v_{i+1})$
 - 6: **end for**
 - 7: **for** $i \in \{n-2, \dots, 1\}$ **do**
 - 8: $(X_i, X_{i+1}) \leftarrow \text{LOCALEFX}(X_i, X_{i+1}, v_i, v_{i+1})$
 - 9: **end for**
 - 10: **if** $X^{\text{init}} = X$ **and** some edge $(i, i+1)$ has strong envy **then**
 - 11: **return** Failure
 - 12: **end if**
 - 13: **end while**
 - return** X
-

4.2 Flowchart of Sweeping Algorithm



4.3 Python Implementation

```

1 def local_efx(x_i, x_j, vi, vj):
2
3     # All goods in M' and sort by vi descending
4     M_prime = sorted(x_i + x_j, key=lambda g: vi[g], reverse=True)
5     Y1, Y2 = [], []
6
7     # Greedy split      always add to the bundle with smaller vi-
8     #                   value
9     for g in M_prime:
10         if sum(vi[x] for x in Y1) <= sum(vi[x] for x in Y2):
11             Y1.append(g)
12         else:
13             Y2.append(g)

```

```

14 # Give the higher-vj bundle to agent j (i+1); agent i gets
    the other
15 if sum(vj[x] for x in Y1) >= sum(vj[x] for x in Y2):
16     return Y2, Y1 # Xi=Y2, X_{i+1}=Y1
17 else:
18     return Y1, Y2 # Xi=Y1, X_{i+1}=Y2
19
20
21
22 def EFX(X_i, X_j, v_i):
23     v_i_Xi = sum(v_i[g] for g in X_i)
24     v_i_Xj = sum(v_i[g] for g in X_j)
25
26     # No envy at all EFX trivially satisfied
27     if v_i_Xi >= v_i_Xj:
28         return True
29
30
31     if not X_j:
32         return True
33
34     # Check if removing any single good from X_j eliminates envy
35     for g in X_j:
36         if v_i_Xi >= v_i_Xj - v_i[g]:
37             return True
38     return False # Envy does not disappear after removing any
    single good
39
40
41 def strong_envy(X_i, X_j, v_i, v_j):
42     return ( not EFX(X_i, X_j, v_i) or not EFX(X_j, X_i, v_j))
43
44
45 def sweeping_algorithm(v, goods):
46     n = len(v) # number of agents
47
48     # Initial allocation
49     X = [[] for _ in range(n)]
50     X[0] = goods[:] # initially, all goods to agent 1
51
52     iteration = 0
53     while True:
54         iteration += 1
55         print(f"\nIteration-_{iteration}")
56
57
58         # Use sorted bundles so comparison is order-independent
59         X_prev = [sorted(bundle) for bundle in X]
60
61         # Forward sweep
62         print("\n---_Forward_Sweep_---")
63         for i in range(n - 1):

```

```

64     X[i], X[i + 1] = local_efx(X[i], X[i + 1], v[i], v[i
        + 1])
65     print(f"\n    local_efx--(Agent_{i+1}, Agent_{i+2}):")
66     for k in range(n):
67         val = sum(v[k][g] for g in X[k])
68         print(f"        Agent_{k+1}: {str(set(X[k])).replace
            ('set()', '{ }')} (value={val})")
69
70     # Backward sweep
71     print("\n---    Backward Sweep    ---")
72     for i in range(n - 2, -1, -1):
73         X[i], X[i + 1] = local_efx(X[i], X[i + 1], v[i], v[i
            + 1])
74         print(f"\n    local_efx--(Agent_{i+1}, Agent_{i+2}):")
75         for k in range(n):
76             val = sum(v[k][g] for g in X[k])
77             print(f"        Agent_{k+1}: {str(set(X[k])).replace
                ('set()', '{ }')} (value={val})")
78
79     # Check whether any adjacent pair still has strong envy
80     print("\n---    Envy Check    ---")
81     strong_envy_exists = False
82     for i in range(n - 1):
83         se = strong_envy(X[i], X[i + 1], v[i], v[i + 1])
84
85         vi, vj = v[i], v[i + 1]
86         val_i_own = sum(vi[g] for g in X[i])
87         val_i_sees = sum(vi[g] for g in X[i + 1])
88         val_j_own = sum(vj[g] for g in X[i + 1])
89         val_j_sees = sum(vj[g] for g in X[i])
90
91         # Least-valued good in opponent's bundle (worst-case
            removal for EFX)
92         min_g_for_i = min(X[i + 1], key=lambda g: vi[g]) if X
            [i + 1] else None
93         min_g_for_j = min(X[i], key=lambda g: vj[g]) if X
            [i] else None
94
95         efx_i = EFX(X[i], X[i + 1], vi)
96         efx_j = EFX(X[i + 1], X[i], vj)
97
98         print(f"\n    Agents_{i+1}&_{i+2}: {'(STRONG_ENVY)' if
            use else '(No strong envy)'}")
99
100        # Agent i's perspective
101        opp_minus_i = f",    opponent_least-valued_good_({
            min_g_for_i}): {vi[min_g_for_i]}" if min_g_for_i
            else ""
102        print(f"        Agent_{i+1} view    own: {val_i_own},
            opponent: {val_i_sees}{opp_minus_i}")
103        print(f"        EFX_{'}' if efx_i else 'NOT'} satisfied
            for Agent_{i+1}.")
104

```

```

105         # Agent j's perspective
106         opp_minus_j = f",_{opp_opponent}_{least-valued}_{good}_{(
            min_g_for_j)}):_{vj[min_g_for_j]}" if min_g_for_j
            else ""
107         print(f"_{Agent}_{i+2}_{view}_{_own}_{val_j_own},_{
            _opponent}_{val_j_sees}{opp_minus_j}")
108         print(f"_{EFX}_{'_'_if_efx_j_else_'NOT'}_{satisfied
            _for_Agent}_{i+2}.")
109
110         if se:
111             strong_envy_exists = True
112
113         # Stopping conditions
114         if X_prev == [sorted(b) for b in X] and
            strong_envy_exists:
115             print("\nFailure:_{Strong}_{envy}_{remains}.")
116             return None
117
118         if not strong_envy_exists:
119             print("\nSuccess:_{Strong}_{envy}_{eliminated}.")
120             return X
121
122
123 # Example
124
125
126
127 import random
128
129 n = int(input('Number_{of}_{agents}_{(n)}:_'))
130 m = int(input('Number_{of}_{goods}_{(m)}:_'))
131
132 goods_list = [f'g{i+1}' for i in range(m)]
133 agents_list = [f'Agent-{i+1}' for i in range(n)]
134 valuations = [
135     {g: random.randint(1, 10) for g in goods_list}
136     for _ in range(n)
137 ]
138 print(f"\nGoods:_{{{','_.join(goods_list)}}}")
139 print(f"\nAgents:_{set(agents_list)}")
140 print('\nValuations:')
141 for i, v in enumerate(valuations):
142     print(f'_{Agent}_{i+1}:_{v}')
143
144 allocation = sweeping_algorithm(valuations, goods_list)
145
146 if allocation:
147     print("\n"+"="*40)
148     print("FINAL_{ALLOCATION}_{")
149     print("="+40)
150     for i, bundle in enumerate(allocation):
151         print(f"Agent_{i+1}_{gets:_{set(bundle)}}")

```

```
print(f"Value: {sum(valuations[i][g] for g in bundle)}")
    )
```

Listing 4.1: Sweeping Algorithm Implementation for Path Graphs

4.3.1 Correctness Analysis

Theorem 5 (Correctness of Sweeping Algorithm [6]). *If the Sweeping Algorithm terminates, then the final allocation produced is a G -EFX allocation for the path graph P_n .*

Proof. The algorithm stops only when there is no pair of neighboring agents who strongly envy each other. In other words, for every edge $(i, i + 1)$ in the path graph, neither agent prefers the other's bundle after removing any single good.

For additive valuations, this condition is exactly the requirement for EFX between two agents. Since the algorithm ensures this condition for every edge in the graph, the final allocation satisfies the G -EFX property.

Therefore, once the algorithm terminates, the allocation is fair (EFX) for all neighboring agents in the path graph. $\square$

4.4 Potential Functions

To study how the algorithm progresses and whether it eventually stops, we consider different potential functions. These functions help us track how the allocation changes over time and whether it is moving toward a stable (envy-free) state [1].

We examine three possible measures:

Table 4.1: Potential functions used to analyze convergence of EFX-type algorithms [1]

Potential Function	Expression	Behavior
Total Envy ϕ_1	$\sum_{i=1}^{n-1} [\text{envy}(i, i + 1) + \text{envy}(i + 1, i)]$	Does not always decrease
Total Strong Envy ϕ_2	$\sum_{i=1}^{n-1} [\text{strong-envy}(i, i + 1) + \text{strong-envy}(i + 1, i)]$	Usually decreases
Minimum Value ϕ_3	$\min_i v_i(X_i)$	Not always consistent

Among these, the total strong envy function (ϕ_2) is the most useful in practice, as it tends to decrease as the algorithm makes progress. However, none of these functions strictly decrease in every step, which makes proving termination more challenging.

Chapter 5

Tree-Based EFX Allocation Algorithm (Proposed)

5.1 Motivation

Paths are special cases of trees. We extend the sweeping approach to arbitrary trees using Depth-First Search (DFS) order.

5.2 Algorithm Description

This algorithm is designed to compute a fair allocation of items among agents who are connected through a tree structure. The goal is to ensure that no agent strongly envies any of their neighbors, which means the final allocation satisfies the EFX property along the edges of the tree.

The process begins by assigning all items to a selected root agent, while all other agents start with empty bundles. The tree is then explored using a depth-first search (DFS), which defines an order for processing the edges.

The algorithm works in repeated rounds. In each round, it performs two main passes over the tree:

- **Forward Pass:** The algorithm moves from the root toward the leaves. For each edge between a parent and child, it redistributes the combined items of the two agents using a local EFX procedure. This step reduces envy in the downward direction.
- **Backward Pass:** Next, the algorithm moves from the leaves back to the root. Again, it applies the local EFX procedure on each edge, this time helping to resolve any remaining envy in the upward direction.

After completing both passes, the algorithm checks whether the allocation has changed. If no changes occur but some envy still exists, the algorithm stops and reports failure. Otherwise, it continues the process until no strong envy remains on any edge.

Once the algorithm terminates successfully, it returns an allocation where fairness (in the EFX sense) is maintained between all neighboring agents in the tree.

This approach builds upon local fairness adjustments between pairs of agents and extends them to a global structure using repeated sweeps over the tree [6].

Algorithm 6 Tree-based EFX Allocation Algorithm

Require: Tree graph $T = (N, E)$, n agents, m items M

Require: Additive valuations $v_i : 2^M \rightarrow \mathbb{R}$

Require: Root $r \in N$

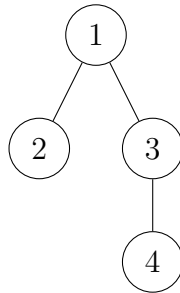
```
1: Initialize:  $X_r \leftarrow M$ ,  $X_i \leftarrow \emptyset$  for all  $i \neq r$ 
2: Compute DFS order:  $[e_1, e_2, \dots, e_{n-1}]$  from root  $r$ 
3: while there exists edge  $(i, j) \in E$  such that  $i$  strongly envies  $j$  do
4:    $X^{init} \leftarrow X$ 
                                      $\triangleright$  Forward pass: root  $\rightarrow$  leaves
5:   for each edge (parent, child) in DFS pre-order do
6:      $(X_{\text{parent}}, X_{\text{child}}) \leftarrow \text{LOCALEFX}(X_{\text{parent}}, X_{\text{child}}, v_{\text{parent}}, v_{\text{child}})$ 
7:   end for
                                      $\triangleright$  Backward pass: leaves  $\rightarrow$  root
8:   for each edge (child, parent) in DFS post-order do
9:      $(X_{\text{child}}, X_{\text{parent}}) \leftarrow \text{LOCALEFX}(X_{\text{child}}, X_{\text{parent}}, v_{\text{child}}, v_{\text{parent}})$ 
10:  end for
11:  if  $X^{init} = X$  and there exists strong envy on some edge then
12:    return Failure
13:  end if
14: end while
15: return  $X$ 
```

5.3 Example of Tree-Based EFX Allocation with Different Valuations

To understand the algorithm more clearly, we consider an example where each agent values goods differently.

5.3.1 Tree Structure

Consider 4 agents arranged in a tree:



5.3.2 Goods and Valuations

Let the set of goods be:

$$M = \{g_1, g_2, g_3, g_4\}$$

Each agent has different additive valuations:

Agent	g_1	g_2	g_3	g_4
1	10	8	6	4
2	7	9	5	3
3	6	4	10	8
4	3	2	7	9

5.3.3 Step 1: Initial Allocation

All goods are given to the root agent:

$$X_1 = \{g_1, g_2, g_3, g_4\}, \quad X_2 = X_3 = X_4 = \emptyset$$

At this stage, all other agents strongly envy agent 1.

5.3.4 Step 2: Forward Sweep

We process edges from root to leaves:

- Edge (1,2): Goods are divided between agents 1 and 2 using LocalEFX.
- Edge (1,3): Remaining goods are fairly split between agents 1 and 3.
- Edge (3,4): Allocation is adjusted between agents 3 and 4.

5.3.5 Step 3: Backward Sweep

Now we move from leaves to root:

- Edge (4,3): Further adjustments are made.
- Edge (3,1): Envy is reduced between agents 3 and 1.
- Edge (2,1): Final corrections are applied.

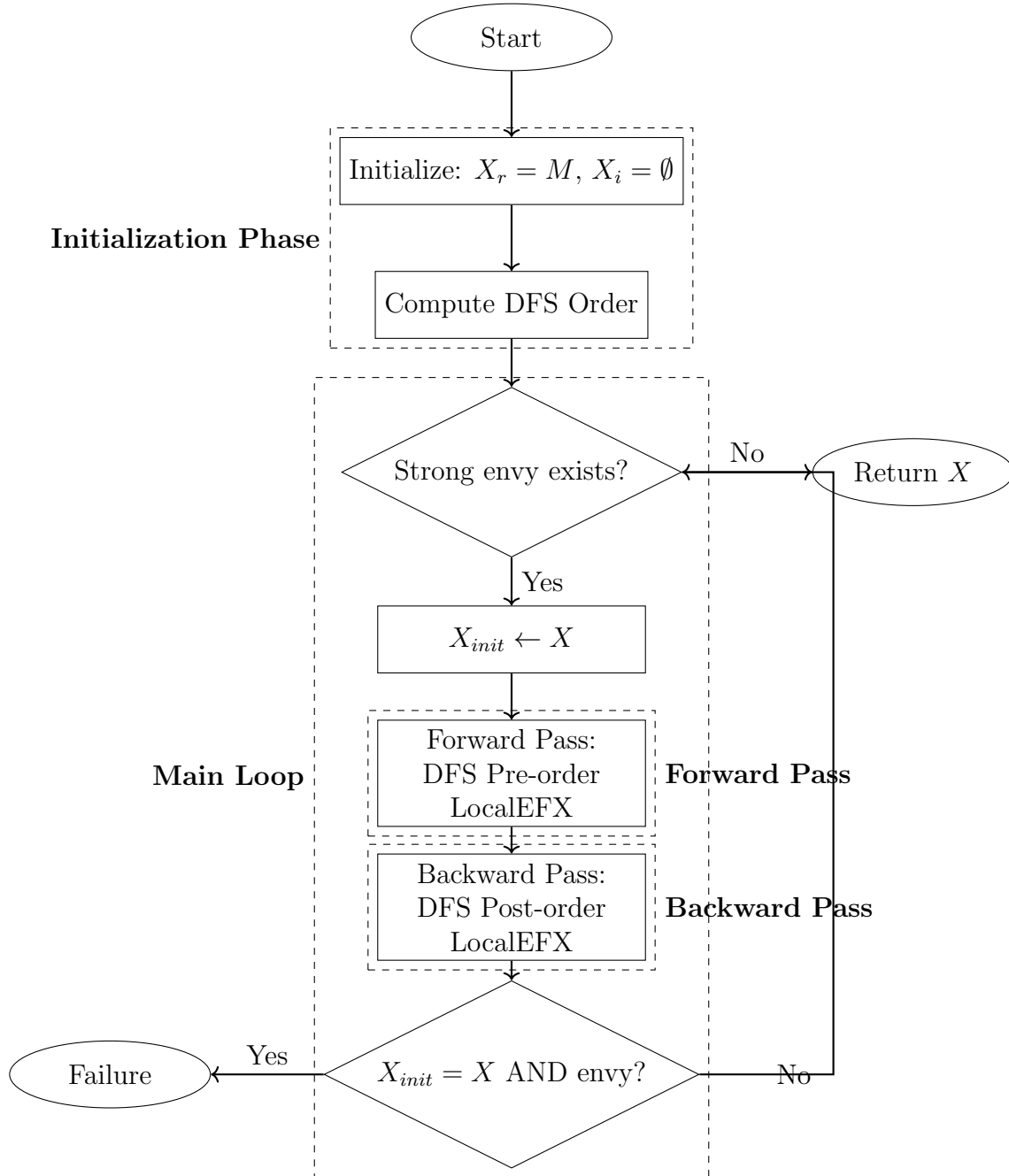
5.3.6 Final Allocation (Example Outcome)

After a few iterations, one possible fair allocation is:

$$X_1 = \{g_1\}, \quad X_2 = \{g_2\}, \quad X_3 = \{g_3\}, \quad X_4 = \{g_4\}$$

Even though agents have different preferences, the algorithm successfully reduces strong envy step by step. By repeatedly applying local fairness adjustments along the edges of the tree, the final allocation satisfies the EFX condition for all neighboring agents.

5.4 Flowchart of Tree-based EFX Allocation Algorithm



5.5 Python Implementation

```

1 import random
2
3
4 # Utility: valuation function
5 def value(v, bundle):
6     return sum(v[g] for g in bundle)
7
8
9

```

```

10 # EFX check
11
12 def EFX(X_i, X_j, v_i):
13     v_i_Xi = value(v_i, X_i)
14     v_i_Xj = value(v_i, X_j)
15
16     if v_i_Xi >= v_i_Xj:
17         return True
18
19     if not X_j:
20         return True
21
22     for g in X_j:
23         if v_i_Xi >= v_i_Xj - v_i[g]:
24             return True
25
26     return False
27
28
29 def strongly_envies(X_i, X_j, v_i):
30     return not EFX(X_i, X_j, v_i)
31
32
33 def strong_envy_pair(X_i, X_j, v_i, v_j):
34     return (not EFX(X_i, X_j, v_i)) or (not EFX(X_j, X_i, v_j))
35
36
37
38 # LocalEFX
39
40 def local_efx(x_i, x_j, vi, vj):
41     M_prime = sorted(x_i + x_j, key=lambda g: vi[g], reverse=True)
42     Y1, Y2 = [], []
43
44     for g in M_prime:
45         if sum(vi[x] for x in Y1) <= sum(vi[x] for x in Y2):
46             Y1.append(g)
47         else:
48             Y2.append(g)
49
50     if sum(vj[x] for x in Y1) >= sum(vj[x] for x in Y2):
51         return Y2, Y1
52     else:
53         return Y1, Y2
54
55
56
57 # DFS traversal (tree)
58
59 def dfs_orders(graph, root):
60

```

```

61     pre_order = []          # List to store edges in pre-order (before
                               # visiting child)
62     post_order = []         # List to store edges in post-order (
                               # after visiting child)
63
64     # Inner recursive DFS function
65     def dfs(node, parent):
66         # Traverse all adjacent nodes (children)
67         for child in graph[node]:
68             # Avoid going back to the parent node
69             if child != parent:
70                 # Record the edge before visiting the child (pre-
                    # order)
71                 pre_order.append((node, child))
72
73                 # Recursive call to visit the child node
74                 dfs(child, node)
75
76                 # Record the edge after returning from the child
                    # (post-order)
77                 post_order.append((child, node))
78
79     # Start DFS from the given root node
80     dfs(root, None)
81
82     # Return both traversal orders
83     return pre_order, post_order
84
85
86 # Check if any strong envy exists
87
88 def exists_strong_envy(graph, X, valuations):
89     for i in graph:
90         for j in graph[i]:
91             if strongly_envies(X[i], X[j], valuations[i]):
92                 return True
93     return False
94
95
96
97 # Print all agents' current bundles
98
99 def print_all_bundles(X, valuations, agents):
100     for k in agents:
101         bundle = set(X[k])
102         val = value(valuations[k], X[k])
103         bundle_str = str(bundle).replace("set()", "{")
104         print(f"Agent {k+1}: {bundle_str} (value={val})")
105
106
107 # Main Algorithm
108
109 def tree_efx_allocation(graph, root, items, valuations):

```

```

110 agents = list(graph.keys())
111
112 # Step 1: Initialization      root gets all goods
113 X = {i: [] for i in agents}
114 X[root] = list(items)
115
116 iteration = 0
117
118 # Loop until no strong envy
119 while exists_strong_envy(graph, X, valuations):
120     iteration += 1
121     print(f"\n{'='*50}")
122     print(f"ITERATION_{iteration}")
123     print("=" * 50)
124
125     X_init = {i: sorted(X[i]) for i in X}
126
127     # Forward pass
128     print("\n---_Forward_Pass_(root_ _leaves)_---")
129     for parent, child in pre_order:
130         X[parent], X[child] = local_efx(
131             X[parent], X[child],
132             valuations[parent], valuations[child]
133         )
134         print(f"\n_{local_efx(Agent_{parent+1},_Agent_{
135             child+1}):")
136         print_all_bundles(X, valuations, agents)
137
138     # Backward pass
139     print("\n---_Backward_Pass_(leaves_ _root)_---")
140     for child, parent in post_order:
141         X[child], X[parent] = local_efx(
142             X[child], X[parent],
143             valuations[child], valuations[parent]
144         )
145         print(f"\n_{local_efx(Agent_{child+1},_Agent_{
146             parent+1}):")
147         print_all_bundles(X, valuations, agents)
148
149     # Envy Check
150     print("\n---_Envy_Check_---")
151     strong_envy_exists = False
152
153     for i in graph:
154         for j in graph[i]:
155             if j <= i:          # check each edge once
156                 continue
157                 vi, vj = valuations[i], valuations[j]
158                 se = strong_envy_pair(X[i], X[j], vi, vj)
159
160                 val_i_own = value(vi, X[i])
161                 val_i_sees = value(vi, X[j])
162                 val_j_own = value(vj, X[j])

```

```

161     val_j_sees = value(vj, X[i])
162
163     min_g_for_i = min(X[j], key=lambda g: vi[g]) if X
164         [j] else None
165     min_g_for_j = min(X[i], key=lambda g: vj[g]) if X
166         [i] else None
167
168     efx_i = EFX(X[i], X[j], vi)
169     efx_j = EFX(X[j], X[i], vj)
170
171     label = "(STRONG_ENVY)" if se else "(No_strong_
172         envy)"
173     print(f"\nAgent_{i+1}&_{j+1}:{label}")
174
175     opp_minus_i = (
176         f",oppoent_least-valued_good_{min_g_for_i
177             }:{vi[min_g_for_i]}"
178         if min_g_for_i else ""
179     )
180     print(f"Agent_{i+1}_view    own:{
181         val_i_own},oppoent:{val_i_sees}{
182             opp_minus_i}")
183     print(f"EFX_{'','if_efx_i}else,'NOT'}_
184         satisfied_for_Agent_{i+1}.")
185
186     opp_minus_j = (
187         f",oppoent_least-valued_good_{min_g_for_j
188             }:{vj[min_g_for_j]}"
189         if min_g_for_j else ""
190     )
191     print(f"Agent_{j+1}_view    own:{
192         val_j_own},oppoent:{val_j_sees}{
193             opp_minus_j}")
194     print(f"EFX_{'','if_efx_j}else,'NOT'}_
195         satisfied_for_Agent_{j+1}.")
196
197     if se:
198         strong_envy_exists = True
199
200 # Stopping condition: no change but envy persists
201 failure
202 if {i: sorted(X[i]) for i in X} == X_init and
203     strong_envy_exists:
204     print("\nFailure: Strong envy remains and allocation
205         is stuck.")
206     return "Failure"
207 if not strong_envy_exists:
208     print("\nSuccess: Strong envy eliminated.")
209     return X
210
211 # ----Generate random tree

```

```

200
201 def generate_random_tree(n):
202     graph = {i: [] for i in range(n)}
203     for i in range(1, n):
204         parent = random.randint(0, i - 1)
205         graph[parent].append(i)
206         graph[i].append(parent)
207     return graph
208
209
210 # Main
211
212 n_agents = int(input("Number of agents (n): "))
213 m_items = int(input("Number of goods (m): "))
214
215 agents = list(range(n_agents))
216
217 # Goods as set {}
218 goods = {f'g{i+1}' for i in range(m_items)}
219
220 # Valuations
221 valuations = {
222     i: {g: random.randint(1, 10) for g in goods}
223     for i in agents
224 }
225
226 # Tree
227 graph = generate_random_tree(n_agents)
228 root = 0
229
230
231 # Print Input
232
233 print("=" * 50)
234 print("INPUT")
235 print("=" * 50)
236
237 print("\nTree (adjacency list):")
238 for node in graph:
239     print(f"Agent {node+1}: {n+1 for n in graph[node]}")
240
241 print("\nGoods:")
242 print(set(sorted(goods, key=lambda g: int(g[1:]))))
243
244 print("\nValuations:")
245 for i in valuations:
246     sorted_val = {g: valuations[i][g] for g in sorted(valuations[i], key=lambda g: int(g[1:]))}
247     print(f"Agent {i+1}: {sorted_val}")
248
249
250 # -----DFS order
251 pre_order, post_order = dfs_orders(graph, root)

```

```

252
253 #-----Print DFS orders
254 print("\n" + "=" * 50)
255 print("DFS TRAVERSAL ORDER")
256 print("=" * 50)
257 print("\nPre-order edges (root → leaves):")
258 for (p, c) in pre_order:
259     print(f"Agent_{p+1} → Agent_{c+1}")
260 print("\nPost-order edges (leaves → root):")
261 for (c, p) in post_order:
262     print(f"Agent_{c+1} → Agent_{p+1}")
263
264 # -----
265 # Allocation Result
266 # -----
267 result = tree_efx_allocation(graph, root, goods, valuations)
268
269
270 if result:
271     print("\n" + "=" * 50)
272     print("FINAL ALLOCATION")
273     print("=" * 50)
274     for i in sorted(result.keys()):
275         bundle = result[i]
276         val = sum(valuations[i][g] for g in bundle)
277         print(f"Agent_{i+1} gets: {set(bundle)}")
278         print(f"Value: {val}")

```

Listing 5.1: Tree-Based EFX Allocation Algorithm

5.6 Correctness

Theorem 6 (Correctness of Tree-Based Algorithm [6]). *If the Tree-Based EFX Allocation Algorithm stops successfully, then the final allocation satisfies the G-EFX property for the tree graph T .*

Proof. The algorithm keeps updating the allocation until there is no strong envy between any pair of neighboring agents in the tree. In other words, for every edge (i, j) , neither agent prefers the other's bundle even after removing any single item.

For additive valuations, this condition is exactly the requirement for EFX between two agents. Since the algorithm ensures this property on every edge of the tree, the final allocation is locally fair across all neighboring pairs.

Therefore, once the algorithm terminates, the allocation satisfies the G-EFX condition for the entire tree. $\square$

5.6.1 Termination Condition

The algorithm may not always stop immediately, as there can be situations where envy keeps shifting between agents in a cyclic manner. However, under additive valuations, the behavior of the algorithm is more controlled.

Each time the LOCALEFX procedure is applied on an edge, one of two things happens:

- either the strong envy between the two agents on that edge is removed, or
- the overall allocation improves according to a chosen measure (for example, the sum of minimum bundle values increases).

Since this improvement cannot continue indefinitely, the algorithm gradually moves toward a stable state. As a result, repeated applications of the local adjustment steps help in reducing envy and pushing the allocation closer to an EFX outcome.

Chapter 6

Conclusion

6.1 Summary of Work

In this project, we studied the problem of fairly dividing indivisible goods among agents when their interactions are restricted by a graph. Our main focus was on the EFX (Envy-Freeness up to Any Good) fairness concept, which is stronger than EF1 and widely studied in fair division.

The work was carried out in two main stages.

In the first stage, we built the theoretical foundation of the problem. We introduced important fairness notions such as EF1, EFX, and Maximum Nash Welfare (MNW). We also reviewed well-known algorithms like Round-Robin and Envy-Cycle Elimination, which guarantee EF1 allocations. In addition, we discussed special cases where EFX allocations are known to exist, such as identical valuations, two-agent settings, and three-agent scenarios [6, 7].

In the second stage, we explored fair division under graph constraints based on recent research [3]. We studied different models such as Graph-HEF and Graph-EFX, and examined how certain graph structures like stars, paths, and trees allow fair allocations even when the general problem remains unresolved.

Based on this understanding, we implemented and analysed two algorithms:

- **Sweeping Algorithm for Path Graphs:** This algorithm starts by assigning all goods to one agent and then repeatedly adjusts allocations along the path using a local two-agent EFX procedure until no strong envy remains.
- **Tree-Based EFX Algorithm:** This is an extension of the sweeping idea to tree graphs. It uses a depth-first traversal and applies local EFX adjustments in both forward and backward directions to remove envy along all edges.

Both algorithms were implemented in Python, and their behavior was carefully tested. We also showed that if these algorithms terminate, they produce allocations that satisfy the G -EFX property.

6.2 Key Findings

The main observations from this project are as follows:

- Limiting comparisons to graph edges makes the problem easier. While global EFX is difficult, graph-based EFX can be achieved for structured graphs like paths, stars, and trees.

- The LocalEFX procedure plays a central role. Since it ensures fairness between two agents, it can be applied repeatedly to build fair allocations in larger graphs.
- Among different measures of progress, total strong envy is the most useful in practice, even though it does not always decrease in every step.
- When both goods and chores are present, EFX may not always exist. However, for graphs with large diameter and structured preferences, fair allocations can still be found [9].
- The tree-based algorithm extends the path-based approach effectively. Although a complete proof of termination is still open, experimental results show that it works well in practice.

6.3 Limitations and Open Problems

There are several important challenges that remain open:

- The termination of both algorithms is not guaranteed in all cases. A strictly decreasing potential function is still unknown.
- The existence of EFX allocations for general graphs under arbitrary valuations is still an open question.
- Extending these results to more complex valuation models, such as submodular preferences, is a difficult problem.
- The computational complexity of checking whether a G -EFX allocation exists is not yet fully understood.

Overall, this project shows that introducing graph structure can make fair division problems more manageable, and it opens several directions for future research.

Bibliography

- [1] R. Mahara, *Extension of Additive Valuations to General Valuations on the Existence of EFX*, Preprint, July 2021.
- [2] J. Payan, R. Sengupta, and V. Viswanathan. "Relaxations of envy-freeness over graphs." *arXiv:2202.10946*, 2022.
- [3] A. Payan, S. Sengupta, and K. Viswanathan, *Graph-Constrained Fair Division*, 2022.
- [4] X. Li, Y. Chen, and Z. Wang. "EFX Allocations of Chores on Graphs." Proceedings of the 2022 ACM Symposium on Theory of Computing, 2022.
- [5] Ioannis Caragiannis, David Kurokawa, Hervé Moulin, Ariel D. Procaccia, Nisarg Shah, and Junxing Wang. *The Unreasonable Fairness of Maximum Nash Welfare*. ACM Transactions on Economics and Computation, 2019.
- [6] Benjamin Plaut and Tim Roughgarden. *Almost Envy-Freeness with General Valuations*. SIAM Journal on Discrete Mathematics, 2020.
- [7] Bhaskar Ray Chaudhury, Jugal Garg, Kurt Mehlhorn, Ruta Mehta, and Pranabendu Misra. *EFX Exists for Three Agents*. Proceedings of the ACM Conference on Economics and Computation (EC), 2020.
- [8] Ryoga Mahara. Existence of EFX allocations for two classes of valuations. *Information Processing Letters*, 174:106295, 2022.
- [9] Hadi Hosseini, Andrew Searns, and Erel Segal-Halevi. Fair division of indivisible goods under constraints. In *Proceedings of the 31st International Joint Conference on Artificial Intelligence (IJCAI)*, pages 292–298, 2022.
- [10] K. Bérczi, E. Csáji, and T. Fleiner, "Fair Division of Mixed Goods and Chores," *Proceedings of the 47th International Colloquium on Automata, Languages, and Programming (ICALP)*, 2020.
- [11] G. Amanatidis, G. Birmpas, E. Markakis, and A. Nikzad. "Multiple-criteria fair division: Complexity and algorithms." In Proceedings of the 35th AAAI Conference on Artificial Intelligence (AAAI), 2021.
- [12] E. Babaioff, N. Nisan, and I. Talgam-Cohen. "Competitive Equilibria with Indivisible Goods and Generic Budgets." *Games and Economic Behavior*, 2021.